

Bancos de Dados

Aula - 05

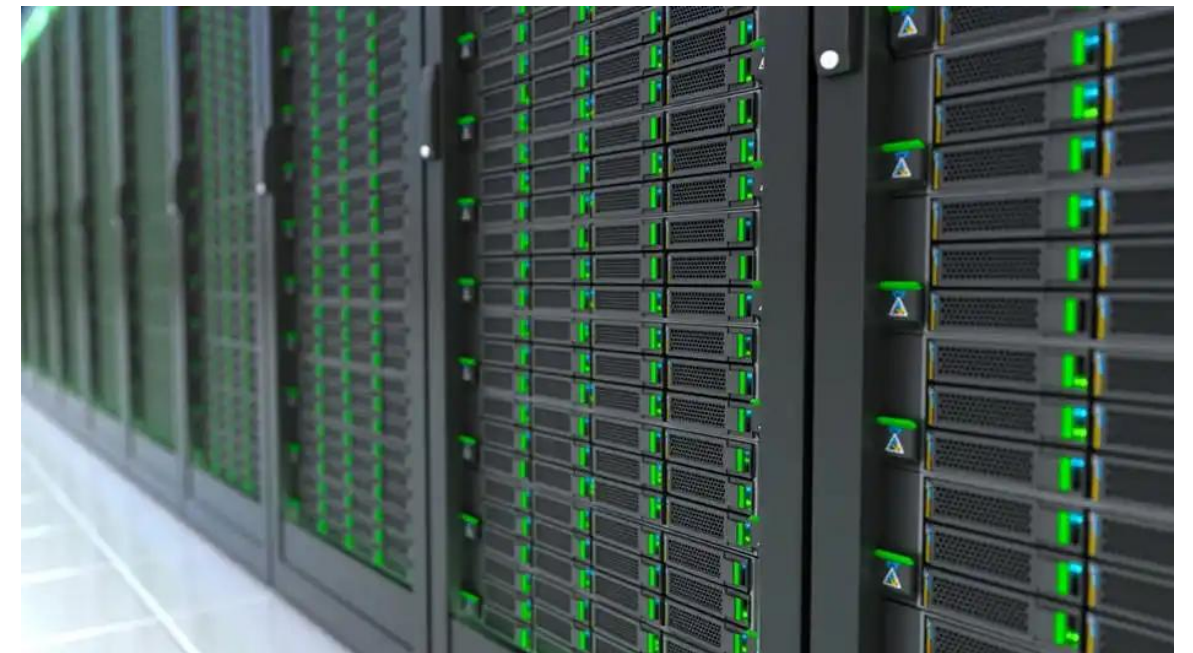
2026

SENAI - CRTI

Consultas e Filtros no MongoDB

Unidade Curricular: Bancos de Dados

Aprenda a recuperar informações precisas em coleções MongoDB utilizando **find()**, **findOne()**, filtros, operadores de comparação, operadores lógicos e projeção — com foco total em prática real.



Agenda da Aula

01

Situação de Aprendizagem

Contextualização do problema real

02

Mobilização de Conhecimentos

Ativação de saberes prévios

03

Desenvolvimento Teórico

Conceitos e exemplos comentados

04

Prática Orientada

10 exercícios progressivos

05

Sistematização + Avaliação

Resumo e atividade final individual



SITUAÇÃO DE APRENDIZAGEM

O Problema: Dados Demais, Informação de Menos

Imagine uma aplicação Node.js conectada a um banco MongoDB com **dezenas de milhares de registros** de usuários, pedidos e produtos. O desenvolvedor precisa encontrar clientes maiores de 18 anos, ativos, de uma determinada cidade — mas não sabe como filtrar corretamente.



Sem domínio de consultas e filtros, o desenvolvedor recupera **todos os documentos** da coleção, consome memória desnecessária e entrega resultados incorretos para a aplicação.

Cenário Real: Sistema de Cadastro

O banco já existe. Os dados estão lá.

A coleção `usuarios` tem milhares de documentos com campos como `nome`, `idade`, `ativo`, `cidade` e `email`.

O desafio do desenvolvedor é **consultar apenas o que precisa**, de forma eficiente e correta, usando os recursos nativos do MongoDB.

Perguntas que surgem no dia a dia:

- Como listar apenas usuários maiores de 18 anos?
- Como buscar apenas usuários ativos E de SP?
- Como retornar somente o nome e e-mail, sem os demais campos?
- Como buscar o primeiro resultado que bate com o filtro?

O Que Você Já Sabe?

Antes de mergulhar nos comandos, vamos ativar o que você já conhece. Reflita sobre as perguntas abaixo — elas conectam experiências anteriores com o conteúdo que vamos aprender hoje.

 Você já filtrou dados?

Em planilhas, SQL ou qualquer outra ferramenta — já precisou mostrar apenas um subconjunto de registros com base em uma condição?

 Como buscar algo específico?

Se você tivesse uma lista de produtos e precisasse encontrar apenas os com preço acima de R\$ 100, como faria isso em código?

 Condições combinadas

Já precisou aplicar mais de uma condição ao mesmo tempo? Por exemplo: preço entre R\$ 50 e R\$ 200 E com estoque disponível?

O Método `find()`

O que faz?

O método `find()` retorna **todos os documentos** de uma coleção que correspondem ao critério de busca fornecido. Sem argumentos, retorna todos os documentos da coleção. É o ponto de entrada para qualquer consulta no MongoDB.

Sintaxe básica

```
db.colecao.find(filtro, projecao)
```

Ambos os parâmetros são opcionais. Quando omitidos, todos os documentos são retornados sem restrição de campos.

Exemplos práticos

```
// Retorna todos os documentos  
db.usuarios.find()
```

```
// Retorna usuários com idade 25  
db.usuarios.find({ idade: 25 })
```

```
// Retorna usuários ativos  
db.usuarios.find({ ativo: true })
```

 O resultado é um **cursor** — uma referência iterável para os documentos encontrados. No terminal do MongoDB Shell, os documentos são listados automaticamente.

O Método `findOne()`

Quando usar?

Use `findOne()` quando precisar recuperar **apenas o primeiro documento** que satisfaz o critério — por exemplo, ao buscar um usuário pelo e-mail (que é único) ou ao verificar a existência de um registro.

Retorna **um objeto** diretamente, não um cursor.

```
db.colecao.findOne(filtro)
```

Comparação direta

<code>find()</code>	<code>findOne()</code>
Retorna cursor	Retorna objeto
Múltiplos docs	Apenas 1 doc
Iterável	Acesso direto

```
// Busca o primeiro usuário ativo
db.usuarios.findOne({ ativo: true })
```

```
// Busca por ID específico
db.usuarios.findOne({
  email: "joao@email.com"
})
```

Filtros: Condições de Busca

Um **filtro** no MongoDB é um documento JSON que especifica as condições que os resultados devem satisfazer. Pense nele como a cláusula **WHERE** do SQL, mas em formato de objeto.

Filtro por valor exato

```
{ nome: "Maria" }
```

Retorna documentos onde o campo **nome** é exatamente "Maria".

Filtro por campo booleano

```
{ ativo: true }
```

Retorna apenas documentos onde o campo **ativo** é verdadeiro.

Filtro por múltiplos campos

```
{ cidade: "SP",  
  ativo: true }
```

Combina condições implicitamente com **AND** — todos os campos precisam bater.

Operadores de Comparação

Quando um valor exato não é suficiente, os **operadores de comparação** do MongoDB permitem definir intervalos, exclusões e igualdades de forma precisa. Eles sempre aparecem dentro de um documento de condição associado a um campo.

\$eq — Igual

```
{ idade: { $eq: 18 } }
```

\$ne — Diferente

```
{ ativo: { $ne: false } }
```

\$gt — Maior que

```
{ idade: { $gt: 18 } }
```

\$gte — Maior ou igual

```
{ idade: { $gte: 18 } }
```

\$lt — Menor que

```
{ preco: { $lt: 100 } }
```

\$lte — Menor ou igual

```
{ preco: { $lte: 100 } }
```

Operadores Lógicos: `$and` e `$or`

`$and` — Todas as condições

Retorna documentos que satisfazem **todas** as condições listadas. É como "isso E aquilo".

```
db.usuarios.find({
  $and: [
    { idade: { $gte: 18 } },
    { ativo: true }
  ]
})
```

Retorna usuários com 18 anos ou mais E que estão ativos.

`$or` — Ao menos uma condição

Retorna documentos que satisfazem **pelo menos uma** das condições. É como "isso OU aquilo".

```
db.usuarios.find({
  $or: [
    { cidade: "SP" },
    { cidade: "RJ" }
  ]
})
```

Retorna usuários de São Paulo OU do Rio de Janeiro.

Projeção: Selecionando Campos

Por padrão, o MongoDB retorna **todos os campos** de cada documento. A **projeção** permite controlar exatamente quais campos aparecem no resultado — útil para economizar memória e deixar as respostas mais limpas na aplicação.

Incluindo campos (**1**)

```
db.usuarios.find(  
  {},  
  { nome: 1, idade: 1 }  
)
```

Retorna apenas `_id`, `nome` e `idade`. O campo `_id` é incluído por padrão, a menos que seja explicitamente excluído.

Excluindo campos (**0**)

```
db.usuarios.find(  
  {},  
  { senha: 0, _id: 0 }  
)
```

Retorna todos os campos **exceto** `senha` e `_id`. Não misture inclusão e exclusão no mesmo documento de projeção (exceto para `_id`).

⚠ Não combine campos com valor **1** e **0** na mesma projeção, exceto para o campo `_id`. Isso gera um erro no MongoDB.

PRÁTICA ORIENTADA

Hora de Praticar!

Os próximos slides apresentam **10 exercícios progressivos**. Cada um introduz um novo conceito ou combina os anteriores. Execute os comandos no terminal do MongoDB Shell e observe os resultados. Use a coleção `usuarios` disponibilizada pelo professor.



EXERCÍCIO 1

Buscar Todos os Documentos

O primeiro passo é confirmar que a coleção existe e visualizar sua estrutura. Este comando retorna **todos os documentos** da coleção sem nenhum filtro aplicado.

```
// Conecte ao banco correto primeiro
use cadastro

// Liste todos os documentos
db.usuarios.find()
```

 No MongoDB Shell, o resultado é paginado. Pressione **it** (iterate) para ver mais resultados caso existam muitos documentos.

O que observar:

- Quais campos cada documento possui?
- Qual é o formato do campo `_id`?
- Quantos documentos existem na coleção?

EXERCÍCIO 2

Buscar por Campo Específico

Agora aplique um **filtro simples** para buscar documentos onde um campo tem um valor exato. Substitua "Maria" por um nome que existe na sua coleção.

```
// Buscar por nome exato
db.usuarios.find({ nome: "Maria" })

// Alternativa com $eq
db.usuarios.find({
  nome: { $eq: "Maria" }
})

// Buscar um único resultado
db.usuarios.findOne({ nome: "Maria" })
```

Desafio extra

Tente buscar por `cidade` ou `email`. O que muda no resultado quando você usa `find()` versus `findOne()`?

Observe que a busca por texto é **sensível a maiúsculas/minúsculas** por padrão no MongoDB.

Filtros com Operadores de Comparação

Aplique operadores para recuperar documentos dentro de **intervalos numéricos**. Esses operadores são fundamentais para filtrar dados como faixas etárias, preços e pontuações.

Exercício 3 — Maior que

```
// Usuários com idade maior que 18
db.usuarios.find({
  idade: { $gt: 18 }
})

// Maior ou igual a 18
db.usuarios.find({
  idade: { $gte: 18 }
})
```

Exercício 4 — Intervalo completo

```
// Idade entre 20 e 30 anos
db.usuarios.find({
  idade: {
    $gte: 20,
    $lte: 30
  }
})

// Diferente de um valor
db.usuarios.find({
  status: { $ne: "inativo" }
})
```

Operadores Lógicos na Prática

Exercício 5 — \$and

Filtre usuários que atendam **todas** as condições simultaneamente:

```
db.usuarios.find({
  $and: [
    { idade: { $gte: 18 } },
    { ativo: true }
  ]
})
```

Também pode ser escrito de forma implícita:

```
db.usuarios.find({
  idade: { $gte: 18 },
  ativo: true
})
```

Exercício 6 — \$or

Filtre usuários que atendam **ao menos uma** das condições:

```
db.usuarios.find({
  $or: [
    { cidade: "São Paulo" },
    { cidade: "Rio de Janeiro" }
  ]
})
```

Compare a quantidade de resultados: o `$or` tende a retornar mais documentos que o `$and`.

Campos Booleanos e Projeção

Exercício 7 – Campo booleano

```
// Apenas usuários ativos
db.usuarios.find({
  ativo: true
})

// Apenas usuários inativos
db.usuarios.find({
  ativo: false
})
```

Conte os resultados de cada consulta. A soma deve ser igual ao total de documentos da coleção.

Exercício 8 – Projeção

```
// Retorna apenas nome e idade
db.usuarios.find(
  {},
  { nome: 1, idade: 1 }
)

// Esconde o _id também
db.usuarios.find(
  {},
  { nome: 1, idade: 1, _id: 0 }
)
```

Observe a diferença no resultado: os campos excluídos pela projeção não aparecem nos documentos retornados.

findOne() e Filtros Combinados

Exercício 9 — findOne()

Utilize `findOne()` para recuperar um único documento. Ideal para busca por identificador único.

```
// Primeiro usuário ativo encontrado
db.usuarios.findOne({
  ativo: true
})

// Busca por email único
db.usuarios.findOne({
  email: "contato@teste.com"
})
```

Exercício 10 — Tudo junto!

Combine filtro, operador lógico e projeção em uma única consulta:

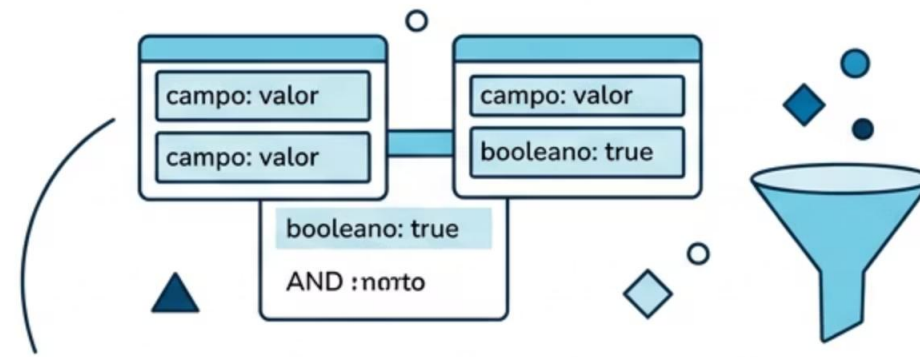
```
db.usuarios.find(
  {
    $and: [
      { idade: { $gte: 18 } },
      { ativo: true },
      { cidade: "São Paulo" }
    ]
  },
  {
    nome: 1,
    email: 1,
    _id: 0
  }
)
```

Resumo Visual dos Comandos

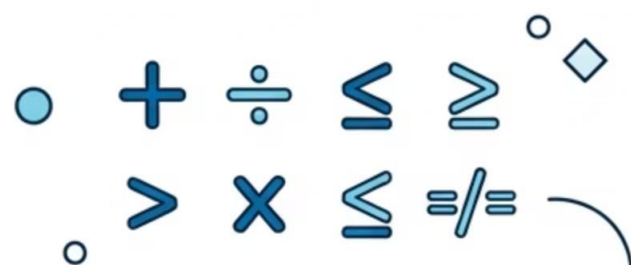
Consolide o que foi aprendido. Este mapa resume os principais recursos de consulta do MongoDB abordados na aula.



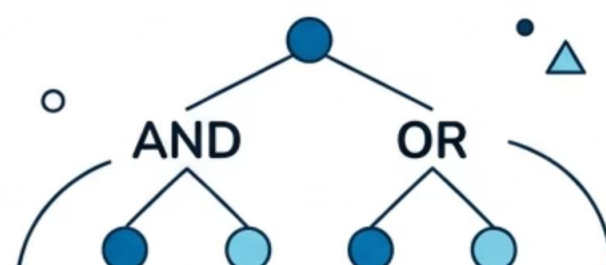
Métodos: `find()` retorna múltiplos, `findOne()` um doc.



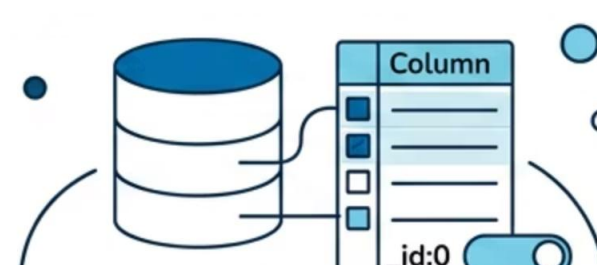
Filtros: Campo=Valor exato, booleanos, AND implícito.



Op. Comparação:
`$eq` igual, `$ne` diferente, `$gt` maior, etc.



Op. Lógicos:
`$and` todas cond.,
`$or` ao menos uma cond.



Projeção:
1 inclui, 0 exclui,
`_id:0` remove ID.

Comparando Consultas: Simples vs. Complexas

Veja como as consultas evoluem de simples para complexas à medida que combinamos recursos. Cada nível adiciona uma camada de precisão ao resultado.

Nível 1 – Sem filtro

```
db.usuarios.find()
```

Retorna todos os documentos. Útil para explorar a coleção.

Nível 2 – Com filtro simples

```
db.usuarios.find({ ativo: true })
```

Filtra por um único campo. Resultado mais preciso.

Nível 3 – Com operador

```
db.usuarios.find({ idade: { $gte: 18 } })
```

Usa operador para definir intervalo ou comparação.

Nível 4 – Filtro + lógica + projeção

```
db.usuarios.find(  
  { $and: [{ idade: { $gte: 18 } }, { ativo: true } ] },  
  { nome: 1, _id: 0 }  
)
```

Consulta completa, profissional e eficiente.

Tabela de Referência Rápida

Use esta tabela como guia de consulta rápida durante os exercícios e a atividade final:

Recurso	Operador/Método	Exemplo
Busca geral	<code>find()</code>	<code>db.col.find()</code>
Busca única	<code>findOne()</code>	<code>db.col.findOne({ email: "x" })</code>
Igual	<code>\$eq</code>	<code>{ idade: { \$eq: 18 } }</code>
Diferente	<code>\$ne</code>	<code>{ ativo: { \$ne: false } }</code>
Maior que	<code>\$gt / \$gte</code>	<code>{ preco: { \$gte: 50 } }</code>
Menor que	<code>\$lt / \$lte</code>	<code>{ preco: { \$lte: 200 } }</code>
E lógico	<code>\$and</code>	<code>{ \$and: [{...}, {...}] }</code>
Ou lógico	<code>\$or</code>	<code>{ \$or: [{...}, {...}] }</code>
Projeção	<code>1 / 0</code>	<code>{ nome: 1, _id: 0 }</code>



ATIVIDADE FINAL

Atividade Prática Individual

Duração: 30 minutos. Esta é a sua oportunidade de demonstrar o que aprendeu. Trabalhe de forma individual utilizando a coleção produtos disponibilizada no banco loja. Registre cada consulta realizada e o resultado obtido.

Tarefas da Atividade Final

Execute todas as consultas abaixo na coleção `produtos` do banco `loja`. Registre os comandos utilizados e os resultados observados para entrega.

1. Buscar todos os documentos

Use `find()` sem filtros para explorar a estrutura da coleção e entender os campos disponíveis.
2. Filtrar por campo específico

Busque produtos de uma categoria específica (ex: `categoria: "eletrônicos"`) usando filtro simples.
3. Aplicar operadores de comparação

Filtre produtos com preço acima de R\$ 100 usando `$gt` e produtos com estoque menor que 10 usando `$lt`.
4. Usar operadores lógicos

Combine condições com `$and` (preço entre R\$ 50 e R\$ 300, disponível) e com `$or` (duas categorias diferentes).
5. Projeção e `findOne()`

Retorne apenas `nome` e `preco` dos produtos filtrados. Use `findOne()` para obter um produto específico pelo nome.

